Fixing invisible / "floating head" characters in The Great Ace Attorney Chronicles

Linux · Steam Proton · Older Intel integrated graphics (tested on Intel HD Graphics 5500, Broadwell)
Latest version, patch file and copyable commands: https://github.com/courierrrrr/tgaac-intel-fix

The problem

On some older Intel laptops running Linux, the 3D characters in *The Great Ace Attorney Chronicles* are mostly missing. You only see floating heads, eyes, hair or props, and the rest of the body is gone or crushed into a flat shape. Backgrounds and text look fine.

The usual tricks don't help: switching Proton versions, DXVK vs. WineD3D, turning off anti-aliasing, changing graphics settings or verifying files. That's because **the bug is in the Intel graphics driver (Mesa), not in the game or Proton.** This guide shows how to build a fixed copy of the driver that only the game uses. Your system drivers stay as they are.



Before: the same frame on the stock Intel driver



After: with the patched driver

Who this is for

Your GPU	Status
Intel HD 5300 / 5500 / 5600 / 6000, Iris 6100 / 6200, Iris Pro P5700 / 6200 (5th gen Core, "Broadwell")	**This guide.** Tested on HD 5500.
Intel HD 400 / 405 (Celeron / Pentium / Atom "Braswell", "Cherry Trail")	Same graphics generation (Gen8). Very likely affected; untested.
Intel HD 4200–5200 (4th gen Core, "Haswell") and older	Uses the same driver code. Might be affected; untested.
Intel 6th gen ("Skylake", HD 520/620…) or newer, AMD, NVIDIA	Not this bug. Don't use this guide.

To check your GPU, run this in a terminal:

```
lspci | grep -i vga
```

Quick way to confirm you have this bug

Add `LIBGL_ALWAYS_SOFTWARE=1 PROTON_USE_WINED3D=1 %command%` to the game's launch options (it renders on the CPU). It will be very slow, but if the characters show up correctly, you have this bug. Remove the option again afterwards.

Why it happens (short version)

Characters are animated with a skeleton. For every point of a character's body, the game reads that bone's position from a small table (a texture) stored on the GPU. It reads the second and third row of each bone as "this cell, then 1 / 2 cells to the right."

The game's coordinates land *exactly* on the border between two cells. On these Intel chips, the hardware's "N cells to the right" lookup in that exact situation grabs the **wrong neighbouring cell**. So parts of the body get another bone's position and are thrown away or flattened. The head and eyes happen to survive, which is why you see floating heads.

The fix changes Mesa's shader compiler for these chips. Instead of letting the hardware apply the "N cells to the right" step, the driver computes the final position itself and moves it 1/256 of a cell inside the correct cell, so the hardware can no longer pick the wrong one. Nothing else changes visually.

What you'll need

- About 2–3 GB of free disk space and 15–40 minutes, mostly waiting for the download and build.
- Steam with Proton. Tested with **GE-Proton11-9**.
- Basic terminal use (copy and paste commands).

Safe to try

Everything goes into one folder in your home directory (`~/mesa-tgaac`). Nothing is installed system-wide, and only this one game uses the patched driver. To undo it, see the last section.

Step 1: Install the build tools

Arch Linux / Manjaro / EndeavourOS (tested)

```
sudo pacman -S --needed base-devel git meson ninja python-mako python-ply \
  python-packaging python-yaml llvm clang libclc spirv-llvm-translator \
  spirv-tools glslang libdrm wayland wayland-protocols libxrandr \
  libxshmfence vulkan-icd-loader
```

Other distributions (untested)

Install Mesa's build dependencies using your package manager, for example `sudo apt build-dep mesa` (Debian/Ubuntu, needs source repositories enabled) or `sudo dnf builddep mesa` (Fedora). You also need `git`, and a recent `meson` (if yours is too old, install it with `pip install --user meson`).

Step 2: Download Mesa and apply the fix

First, create the patch file: copy the **whole command from Appendix A** (from `base64` down to the final `EOF`), paste it into a terminal and press Enter. It writes `~/elk-vtx-offset.patch`. Then check the file arrived intact:

```
sha256sum ~/elk-vtx-offset.patch
```

The result must start with `6c642649df99b8ca`. If it doesn't, copy the command again. Then run:

```
mkdir -p ~/mesa-tgaac && cd ~/mesa-tgaac
git clone https://gitlab.freedesktop.org/mesa/mesa.git src
cd src
git checkout 335d5698dca1d1b9d11ef18cf5f523cc0f7f58fb
git apply --ignore-whitespace ~/elk-vtx-offset.patch
```

The `git checkout` line selects the exact Mesa version this patch was tested on. If `git apply` prints nothing, the patch applied correctly.

Step 3: Build the Vulkan driver

This only builds the Intel Vulkan driver for older chips ("hasvk"), so it's much faster than building all of Mesa.

```
cd ~/mesa-tgaac/src
meson setup ../build -Dbuildtype=release -Dprefix=$HOME/mesa-tgaac/install \
  -Dvulkan-drivers=intel_hasvk -Dgallium-drivers= -Dplatforms=x11,wayland \
  -Dglx=disabled -Degl=disabled -Dgles1=disabled -Dgles2=disabled \
  -Dopengl=false -Dvideo-codecs= -Dintel-rt=disabled
ninja -C ../build install
```

When it finishes, this file must exist (the next steps use it):

```
ls ~/mesa-tgaac/install/share/vulkan/icd.d/intel_hasvk_icd.x86_64.json
```

Step 4: Put DXVK 2.5.1 in the game folder

New DXVK versions (2.6 and later, including the one inside current Proton) refuse to run on these older Intel chips. Version 2.5.1 works, so the game gets its own copy:

```
cd ~/mesa-tgaac
curl -LO https://github.com/doitsujin/dxvk/releases/download/v2.5.1/dxvk-2.5.1.tar.gz
tar xf dxvk-2.5.1.tar.gz
```

Next, find the game folder: in Steam, right-click the game → **Manage** → **Browse local files**. It's usually `~/.local/share/Steam/steamapps/common/TGAAC`. Copy the two 64-bit DLLs into it:

```
cp ~/mesa-tgaac/dxvk-2.5.1/x64/d3d11.dll ~/mesa-tgaac/dxvk-2.5.1/x64/dxgi.dll \
   ~/.local/share/Steam/steamapps/common/TGAAC/
```

Step 5: Set up Steam

1. Right-click the game → **Properties** → **Compatibility**. Tick *Force the use of a specific Steam Play compatibility tool* and choose **GE-Proton** (install it with ProtonUp-Qt if you don't have it).

2. Create a small launcher script that tells the game to use the patched driver and the DXVK files. Paste this whole block into a terminal:

```
cat > ~/mesa-tgaac/run <<'EOF'
#!/bin/sh
ICD=$HOME/mesa-tgaac/install/share/vulkan/icd.d/intel_hasvk_icd.x86_64.json
export VK_DRIVER_FILES=$ICD VK_ICD_FILENAMES=$ICD
export WINEDLLOVERRIDES="d3d11,dxgi=n,b"
exec "$@"
EOF
chmod +x ~/mesa-tgaac/run
```

What it does:

- `VK_DRIVER_FILES` and `VK_ICD_FILENAMES` point Vulkan at your patched driver. The two do the same job; old and new Vulkan loaders read different names, so both are set.
- `WINEDLLOVERRIDES` makes the game use the DXVK 2.5.1 files from step 4.

3. In **General** → **Launch Options**, enter the following, replacing `YOURNAME` with your Linux username (run `whoami` to see it):

```
/home/YOURNAME/mesa-tgaac/run %command%
```

4. Launch the game and go into any scene with characters. They should now have full bodies.

Troubleshooting

Check what the game is actually using

Change the launch options to `PROTON_LOG=1 /home/YOURNAME/mesa-tgaac/run %command%` and start the game once. Then run:

```
grep -E "DXVK: v|Driver *:" ~/steam-1158850.log
```

You should see `DXVK: v2.5.1` and a Mesa version of **26.99** or higher. Remove `PROTON_LOG=1` afterwards.

Symptom	Fix
Game doesn't start, or log says no Vulkan device	Check that `YOURNAME` in the launch options is right, that `~/mesa-tgaac/run` exists, and that the `.json` file from step 3 exists.
Log shows a different DXVK version	The DLLs aren't in the game folder. Copy them again (step 4).
Fixed at first, but broken again after "Verify integrity of game files" or a game update	Steam removed the DXVK DLLs. Copy them again (step 4).
Characters still broken, log looks right	

	Your GPU may have a different bug. Please post your GPU model and log.

Undo everything

1. Clear the launch options in Steam.
2. Delete `d3d11.dll` and `dxgi.dll` from the game folder, then run **Verify integrity of game files**.
3. Delete the `~/mesa-tgaac` folder (it also contains the launcher script).

You don't need to update this driver copy for this game. Your normal system drivers keep updating as usual. If the fix is accepted into official Mesa, this guide won't be needed any more.

Appendix A: command that creates the patch file

Copying code out of a PDF scrambles its spacing, which breaks patches. So the patch is encoded here as letters and numbers (base64), which survive copy and paste. The command decodes it back into the exact file shown in Appendix B.

```
base64 -d -i > ~/elk-vtx-offset.patch <<'EOF'
ZGlmZiAtLWdpdCBhL3NyYy9pbnRlbC9jb21waWxlci9lbGsvZWxrX25pci5jIGIvc3JjL2ludGVs
L2NvbXBpbGVyL2Vsay9lbGtfbmlyLmMKaW5kZXggODI2NjQ0My4uN2I3MmY1MCAxMDA2NDQKLS0t
IGEvc3JjL2ludGVsL2NvbXBpbGVyL2Vsay9lbGtfbmlyLmMKKysrIGIvc3JjL2ludGVsL2NvbXBp
bGVyL2Vsay9lbGtfbmlyLmMKQEAgLTEsMyArMSw1IEBACisjaW5jbHVkZSAiY29tcGlsZXIvbmly
L25pcl9idWlsdGluX2J1aWxkZXIuaCIKKyNpbmNsdWRlICJ1dGlsL3VfZGVidWcuaCIKIC8qCiAg
KiBDb3B5cmlnaHQgwqkgMjAxNCBJbnRlbCBDb3Jwb3JhdGlvbgogICogU1BEWC1MaWNlbnNlLUlk
ZW50aWZpZXI6IE1JVApAQCAtOTIyLDYgKzkyNCw1NSBAQCBsb3dlcl94ZWhwX3RnNF9vZmZzZXRf
ZmlsdGVyKGNvbnN0IG5pcl9pbnN0ciAqaW5zdHIsIFVOVVNFRCBjb25zdCB2b2lkICpkYXRhKQog
ICAgcmV0dXJuIG9mZnNldF94IDwgLTggfHwgb2Zmc2V0X3ggPiA3IHx8IG9mZnNldF95IDwgLTgg
fHwgb2Zmc2V0X3kgPiA3OwogfQogCisvYXJnIGNoYXJhY3RlcnMgbG9zZSBtb3N0IG9m
IHRoZWlyIGdlb21ldHJ5KS4gRm9sZCB0aGUgb2Zmc2V0IGludG8gdGhlCisgKiBjb29yZGluYXRl
IGFuZCBudWRnZSBpdCAxLzI1NiBvZiBhIHRleGVsIG2ggaXMgYmVsb3cgdGhl
CisgKiBzYW1wbGVyJ3Mgc3ViLXRleGVsIHByZWNpc2lvbiBhbmQgdGhlcmVmb3JlIGludmlzaWJs
ZSBmb3IgbGluZWFyIGZpbHRlcmluZy4KKyAqIERpc2FibGUgd2l0aCBFTEtfS0VFUF9WVFhfVEVY
X09GRlNFVD0xLgorICovCitzdGF0aWMgYm9vbAorbG93ZXJfdnR4X3RleF9vZmZzZXRfbnVkZ2Uo
bmlyX2J1aWxkZXIgKmIsIG5pcl90ZXhfaW5zdHIgKnRleCwgdm9pZCAqZGF0YSkKK3sKKyAgIGlm
ICh0ZXgtPm9wICE9IG5pcl90ZXhvcF90ZXggJiYgdGV4LT5vcCAhPSBuaXJfdGV4b3BfdHhsICYm
CisgICAgICAgdGV4LT5vcCAhPSBuaXJfdGV4b3BfdHhiICYmIHRleC0+b3AgIT0gbmlyX3RleG9w
X3R4ZCkKKyAgIHJldHVybiBmYWxzZTsKKworICAgaWYgKHRleC0+c2FtcGxlciAhPSAtMSkKKyAg
ICAgICByZXR1cm4gZmFsc2U7CisKKyAgIHJldHVybiB0cnVlOworfQorCitzdGF0aWMgYm9vbAor
bHNlOworCisgICBuaXJfZGVmICpvZmZzZXQgPSBuaXJfc3RlYWxfdGV4X3NyYyh0ZXgsIG5pcl90
ZXhfc3JjX29mZnNldCk7CisgICBpZiAoIW9mZnNldCkKKyAgICAgICByZXR1cm4gZmFsc2U7Cisg
ICBuaXJfX2RlZiAqY29vcmQgPSBuaXJfc3RlYWxfdGV4X3NyYyh0ZXgsIG5pcl90ZXhfc3JjX2Nv
b3JkKTsKKyAgIG5pcl9pbnN0ciAqaW5zID0gbmlyX2NvbnN0cigmd2hpY2ggdGV4LT5vcCAhPSBu
ICh0ZXgtPm9wICE9IG5pcl90ZXhvcF90ZXggJiYgdGV4LT5vcCAhPSBuaXJfdGV4b3BfdHhsICYm
CisgICAgICAgdGV4LT5vcCAhPSBuaXJfdGV4b3BfdHhiICYmIHRleC0+b3AgIT0gbmlyX3RleG9w
X3R4ZCkKKyAgICAgICBuaXJfX2RlZiAqY29vcmQgPSBuaXJfc3RlYWxfdGV4X3NyYyh0ZXgsIG5p
cl90ZXhfc3JjX2Nvb3JkKTsKKyAgICAgICBuaXJfaW5zX2F0KGJsb2NrX2VuZCh0ZXgtPmJiKSwg
ICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICA
```

Appendix B: the patch, readable (`elk-vtx-offset.patch`)

This is what the command in Appendix A creates, so you can see exactly what changes. Lines starting with `+` are added code. It only touches one file of Mesa's Intel shader compiler, and only for non-pixel shaders. Setting `ELK_KEEP_VTX_TEX_OFFSET=1` turns it off.

```
diff --git a/src/intel/compiler/elk/elk_nir.c b/src/intel/compiler/elk/elk_nir.c
index 8266443..7b72f50 100644
--- a/src/intel/compiler/elk/elk_nir.c
+++ b/src/intel/compiler/elk/elk_nir.c
@@ -1,3 +1,5 @@
+#include "compiler/nir/nir_builtin_builder.h"
+#include "util/u_debug.h"
 /*
  * Copyright © 2014 Intel Corporation
  * SPDX-License-Identifier: MIT
@@ -922,6 +924,55 @@ lower_xehp_tg4_offset_filter(const nir_instr *instr, UNUSED const void *data)
    return offset_x < -8 || offset_x > 7 || offset_y < -8 || offset_y > 7;
 }

+/* Gfx8 (and older) samplers pick the wrong texel when a non-fragment-stage
+ * sample with a texel offset lands exactly on a texel edge with nearest
+ * filtering (seen in The Great Ace Attorney Chronicles' bone-matrix fetches,
+ * where characters lose most of their geometry). Fold the offset into the
+ * coordinate and nudge it 1/256 of a texel inward, which is below the
+ * sampler's sub-texel precision and therefore invisible for linear filtering.
+ * Disable with ELK_KEEP_VTX_TEX_OFFSET=1.
+ */
+static bool
+lower_vtx_tex_offset_nudge(nir_builder *b, nir_tex_instr *tex, void *data)
+{
+   if (tex->op != nir_texop_tex && tex->op != nir_texop_txl &&
+       tex->op != nir_texop_txb && tex->op != nir_texop_txd)
+      return false;
+
+   if (tex->sampler_dim == GLSL_SAMPLER_DIM_RECT)
+      return false;
+
+   int coord_index = nir_tex_instr_src_index(tex, nir_tex_src_coord);
+   if (coord_index < 0 ||
+       nir_tex_instr_src_type(tex, coord_index) != nir_type_float)
+      return false;
+
+   nir_def *offset = nir_steal_tex_src(tex, nir_tex_src_offset);
+   if (!offset)
+      return false;
+
+   nir_def *coord = tex->src[coord_index].src.ssa;
+
+   b->cursor = nir_before_instr(&tex->instr);
+
+   nir_def *txs = nir_i2f32(b, nir_build_texture_query(b, tex, nir_texop_txs,
+                                                       offset->num_components,
+                                                       nir_type_int32,
+                                                       false, true));
+   nir_def *shift = nir_fadd_imm(b, nir_i2f32(b, offset), 1.0 / 256.0);
+   nir_def *offset_coord = nir_fadd(b, coord,
+                                    nir_fmul(b, shift, nir_frcp(b, txs)));
+
+   if (tex->is_array) {
+      unsigned a = tex->coord_components - 1;
+      offset_coord = nir_vector_insert_imm(b, offset_coord,
+                                           nir_channel(b, coord, a), a);
+   }
+
+   nir_src_rewrite(&tex->src[coord_index].src, offset_coord);
+   return true;
```

```
+}
+
 static bool
 lower_txd_cb(const nir_tex_instr *tex, const void *data)
 {
@@ -1013,6 +1064,11 @@ elk_preprocess_nir(const struct elk_compiler *compiler, nir_shader *nir,
        .lower_txd_data = devinfo,
     };

+   if (nir->info.stage != MESA_SHADER_FRAGMENT &&
+       !debug_get_bool_option("ELK_KEEP_VTX_TEX_OFFSET", false))
+      OPT(nir_shader_tex_pass, lower_vtx_tex_offset_nudge,
+          nir_metadata_control_flow, NULL);
+
     OPT(nir_lower_tex, &tex_options);
     OPT(nir_normalize_cubemap_coords);
```